Mandelbrot Set

Preface

The following are notes on my attempt on rendering the Mandelbrot set with high precision.

The Mandelbrot set

The Mandelbrot set's definition can be written in multiple ways.

> Consider iterations of a quadratic polynomial F over complex numbers:
>
> $$F(z, c) = z^2 + c F^{n+1}(z, c) = F(F^n(z, c), c)$$
>
> The Mandelbrot set M is the set of $c \in \mathbb{C}$ for which the iterates of $z = 0$ remain bounded:
>
> $$M = \{c \in \mathbb{C} : F^n(0, c) \nrightarrow \infty \text{ as } n \to \infty\}$$
>
> — Perturbation techniques applied to the Mandelbrot set

Visualizing the Mandelbrot set

Despite its simple formula, visualizing the Mandelbrot set requires more techniques than simple brute-forcing, the further we zoom into the fractal.

Naive implementation

A naive attempt of visualizing the Mandelbrot set can look like this:

```
vec2 mult(vec2 v1, vec2 v2) {
    return vec2(
        v1.x * v2.x - v1.y * v2.y, // Real part
        v1.x * v2.y + v1.y * v2.x // Imaginary part
    );
}

float compute_iterations(vec2 c) {
    const float B = 256.0f;
    float n = 0.0f;
    vec2 z = vec2(0.0f);

    for (int i = 0; i < max_iter; i++) {
        z = mult(z, z) + c;
        if (dot(z, z) > B * B) break;
        n += 1.0f;
    }

    // Smooth iteration count
    float sn = n - log2(log2(dot(z,z))) + 4.0;
    return sn;
}
```

where `sn` (smooth iteration count) can be used to assign a specific color to the corresponding value of c.

However, if we try to zoom in too far the limitation on floating point precision is quite apparent.

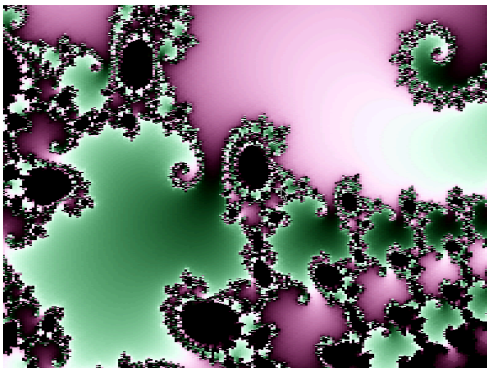

Figure 1: Unclear render of a fractal due to imprecision of floating point numbers.

Perturbation techniques

The following explanation is adapted from Deep zoom theory and practice by Claude Heiland-Allen.

To mitigate this limitation, we must use higher precision floating point types for the calculation process. However, calculating the appropriate iteration count for each high precision floating point types is most likely too slow.

To speed up this process, we introduce perturbation techniques.

The core idea is quite simple. Since $z^2 + c$ is analytic, nearby points remain nearby under iteration. Let upper case variables (Z, C) be the unperturbed reference (in high precision) and lower case variables (z, c) be the perturbed per pixel delta (in low precision).

Start with the iteration formula:

$$Z \to Z^2 + C$$

Perturb the variables:

$$(Z + z) \to (Z + z)^2 + (C + c) = (Z^2 + C) + (2Zz + z^2 + c)$$

Then

$$z \to 2Zz + z^2 + c.$$

We can call the sequence of values the reference point (Z) takes under $z^2 + c$ a *reference orbit* and the sequences of values other perturbed value (z) takes a *delta orbit*.

The idea is:

1. Choose a reference point (in high precision).
2. Calculate the reference orbit
   - The orbit calculation is done in high precision, but the resulting value can be stored as low precision `float`.
3. For other points, use the reference orbit to calculate their iteration counts.

We can calculate reference orbits like following:

```
void ReferenceOrbit::calculate_reference(const std::complex<double> &center) {
  // Remove previous orbit values
  orbit.clear();

  std::complex<long double> z{0};
  for (int i{0}; i < conf::kMaxIteration; ++i) {
    orbit.push_back(static_cast<std::complex<float>>(z));
    z = z * z + static_cast<std::complex<long double>>(center);
```

```
    if (std::norm(z) > conf::kFractalExitBoundary)
      break;
  }
}
```

Of course, `long doule` may not be enough for a higher zoom level.

With the reference orbit in hand, for any other point, we can calculate their iteration count as follows:

```
float compute_iterations_ref(vec2 dc) {
    const float B = 256.0f;
    vec2 dz = vec2(0.0f);
    int i = 0;
    for (i = 0; i < max_iter && i < orbit.length() - 1; i++) {
        dz = 2.0f * mult(orbit[i], dz) + mult(dz, dz) + dc;
        if (dot(orbit[i + 1] + dz, orbit[i + 1] + dz) > B * B) break;
    }
    if (i == max_iter || i + 1 == orbit.length())
        return 0.0f;
    float sn = float(i) - log2(log2(dot(orbit[i] + dz, orbit[i] + dz))) + 4.0;
    return sn;
}
```

Unfortunately, this strategy fails when the reference point chosen has a low iteration count. Naturally since this is an approximation, it starts to fail when the delta orbit stays too far away from the reference orbit or when a bunch of delta orbit obtains the same iteration count.

A bunch of delta orbit can have the same iteration count when the reference orbit exits (finishes) too quickly:

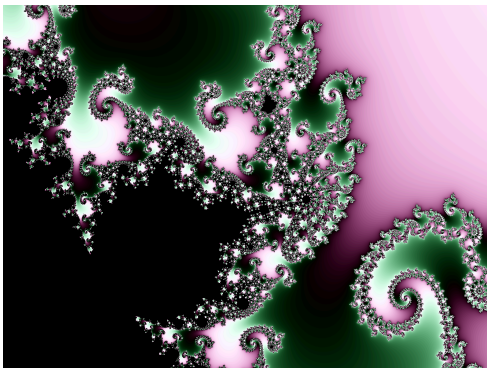

Figure 2: Perturbation technique applied with “good” reference point.

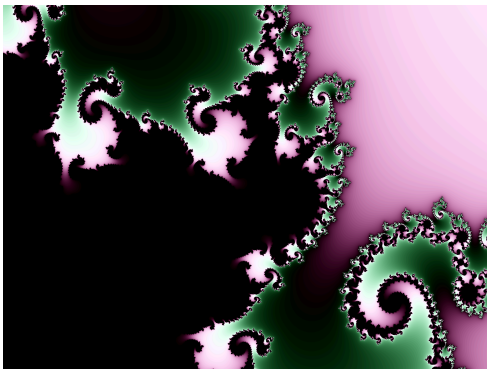

Figure 3: Color clumps originating from many delta orbit with the same iteration count.

Of course, if we face such situation, we can choose a new (hopefully better) reference orbit and recalculate the delta orbit.

Zhuoran from fractal forums suggested the following approach, which actually avoids the need to choose a new reference orbit:

```
float compute_iterations_ref(vec2 dc) {
    const float B = 256.0f;
    vec2 dz = vec2(0.0f);
    int iter;
    int reference_iter = 0;
    int max_reference_iter = orbit.length();
    for (iter = 0; iter < max_iter; iter++) {
        dz = 2.0f * mult(orbit[reference_iter], dz) + mult(dz, dz) + dc;
        reference_iter++;

        vec2 z = orbit[reference_iter] + dz;
        if (dot(z, z) > B * B) break;
        if (dot(z, z) < dot(dz, dz) || reference_iter == max_reference_iter) {
            dz = z;
            reference_iter = 0;
        }
    }
    float sn = float(iter) - log2(log2(dot(orbit[iter] + dz, orbit[iter] + dz))) +
4.0;
    return sn;
}
```